Notebook for Programming in Problem 4

Welcome to the programming portion of the assignment! Each assignment throughout the semester will have a written portion and a programming portion. We will be using Google Colab, so if you have never used it before, take a quick look through this introduction: Working with Google Colab.

Cross Entropy Loss on GPU

In this problem you will benchmark the cross-entropy loss in PyTorch and measure its efficiency relative to the theoretical memory bandwidth of the GPU. You will predict performance before measuring, then explain any discrepancies — this is the core skill of systems performance analysis. Setup: Use the following parameters throughout:

- Batch size: B = 4,096
- Vocabulary size: V = 128,256
- Logits dtype: torch.bfloat16
- Reduction: 'none'
- Device: GPU (use a Colab GPU runtime, e.g., T4 or A100)
- Report your PyTorch version (e.g., torch. version) and GPU model in your notebook. Results can vary significantly across versions.

Generate random input logits and targets

```
# Generate random input logits and targets:
import torch
B, V = 4096, 128256
logits = torch.randn(B, V, dtype=torch.bfloat16 , device='cuda',
requires_grad=True)
targets = torch.randint(0, V, (B,), device='cuda')
```

Questions

(a) Predict before you measure. (8 points)

Before writing any benchmarking code, use your answers from Theory Problem 1 to predict the performance of an ideal (perfectly memory-bound) cross-entropy kernel on your GPU.

(i) Look up the peak memory bandwidth of your Colab GPU. For this assignment, if you are using an A100, assume the A100 SXM bandwidth of ~2039 GB/s; if you are using a T4, use ~300 GB/s. Using the total bytes from Theory Problem 1(c), compute the minimum possible time (in ms) for the forward pass and backward pass, assuming the kernel runs at 100% of peak memory bandwidth. (4 points)

(ii) Based on your arithmetic intensity analysis from Theory Problem 1(c), explain why memory bandwidth (not TFLOPS) is the right performance limiter and the right metric to evaluate the efficiency of cross- entropy. (4 points)

(i) For an A100, assume peak memory bandwidth is about 2039 GB/s. From Theory part 1(c)(iii), the total memory traffic is:

- Forward: 1.0507 GB

- Backward: 2.1014 GB

Minimum possible time, Using: $time(ms)$ = $(\frac{bytes}{bandwidth}) \times 1000$

- Forward: $(\frac{1.0507}{2039}) \times 1000 \approx 0.515\ ms$
- Backward: $(\frac{2.1014}{2039}) \times 1000 \approx 1.031\ ms$

(ii) Why memory bandwidth is the limiter

From Theory Part 1(c)(iv), arithmetic intensity is very low:

- Forward ≈ $1.5\ FLOPs/byte$
- Backward ≈ $1.5\ FLOPs/byte$
- These are much smaller than the A100 compute-to-bandwidth ratio (~153 FLOPs/byte), so the kernel cannot utilize full compute capacity.
- Cross-entropy is memory-bound, meaning performance is limited by how fast data is moved between HBM and the GPU, not by compute (TFLOPS). Therefore, memory bandwidth (GB/s) is the correct metric to evaluate efficiency, rather than peak compute performance.

(b) Benchmark PyTorch eager mode. (12 points)

(i) Write a benchmarking function that measures the wall-clock time (in milliseconds) for the cross-entropy forward pass and backward pass separately. Use torch.cuda.Event with enable_timing=True for accurate GPU timing. After recording the end event, make sure to synchronize the GPU (e.g., with torch.cuda.synchronize() or end_event.synchronize()) before reading the elapsed time. Run each measurement multiple times (e.g., 100 iterations) and report the median time. Make sure to do a few warmup iterations before timing. (5 points)

(ii) Compute the achieved memory bandwidth (in GB/s) for the forward and backward passes: Achieved BW (GB/s) = Total bytes (GB) / Time (s)

How does the measured time compare to your prediction from part (a)? If there is a significant gap, explain why PyTorch eager mode is slower than the ideal. What extra memory traffic or overhead might the eager implementation incur? (7 points)

Hint: Think about whether the eager implementation fuses all the operations into a single kernel, or whether it materializes intermediate tensors (e.g., the full softmax output) to HBM.

(ii) Using Achieved BW = $(\frac{Total\ Bytes}{Time})$

- Forward BW = $(\frac{1.0507}{3.05077/1000})$ ≈ $343.62\ GB/s$
- Backward BW = $(\frac{2.1014}{3.7642/1000})$ ≈ $558.26\ GB/s$

How does the measured time compare to your prediction from part (a)?

Compared to the ideal prediction from part (a), there is a significant gap.

- Ideal forward time: $0.515\ ms$
- Ideal backward time: $1.031\ ms$

the measured eager times are much slower:

- Measured forward: $3.0577\ ms$
- Measured backward: $3.7642\ ms$

So eager mode is about $\frac{3.0577}{0.515} \approx 5.9\times$ slower for forward and $\frac{3.7642}{1.031} \approx 3.7\times$ slower for backward. A likely reason is that PyTorch eager mode does not fully fuse all the cross-entropy operations into one ideal kernel. Instead, it may launch multiple kernels and materialize intermediate tensors in HBM. This increases total memory traffic compared to the ideal byte count.

For example, softmax related intermediate results may be written to memory and read again later. There is also kernel launch overhead due to multiple launching of kernals. Because cross-entropy is memory-bound, this extra memory movement makes the eager implementation slower than the ideal prediction.

Code for part (b):

```
import torch
import torch.nn.functional as F
import statistics

# Taken from part theory part
BYTES_FWD_GB = 1.0507
BYTES_BWD_GB = 2.1014

def benchmark_ce_eager(logits, targets, iters=100, warmup=10):
    fwd_times = []
    bwd_times = []

    # Warmup
    for _ in range(warmup):
        loss = F.cross_entropy(logits, targets, reduction='none')
        grad_out = torch.ones_like(loss, dtype=torch.float32)
        loss.backward(grad_out, retain_graph=False)
        logits.grad = None
    torch.cuda.synchronize()

    # Forward benchmark
    for _ in range(iters):
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)

        start.record()
        loss = F.cross_entropy(logits, targets, reduction='none')
        end.record()

        torch.cuda.synchronize()
        fwd_times.append(start.elapsed_time(end))  # ms

    # Backward benchmark
    for _ in range(iters):
        loss = F.cross_entropy(logits, targets, reduction='none')
        grad_out = torch.ones_like(loss, dtype=torch.float32)

        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)

        logits.grad = None
        start.record()
```

```
            loss.backward(grad_out, retain_graph=False)
            end.record()

            torch.cuda.synchronize()
            bwd_times.append(start.elapsed_time(end))  # ms

        fwd_median_ms = statistics.median(fwd_times)
        bwd_median_ms = statistics.median(bwd_times)

        return fwd_median_ms, bwd_median_ms, fwd_times, bwd_times


# Run benchmark
fwd_ms, bwd_ms, fwd_times, bwd_times = benchmark_ce_eager(logits, targets, iters=100, warmup=10)

# Achieved bandwidth
fwd_bw = BYTES_FWD_GB / (fwd_ms / 1000.0)
bwd_bw = BYTES_BWD_GB / (bwd_ms / 1000.0)

print(f"Eager forward median time:  {fwd_ms:.4f} ms")
print(f"Eager backward median time: {bwd_ms:.4f} ms")
print(f"Eager forward achieved BW:  {fwd_bw:.2f} GB/s")
print(f"Eager backward achieved BW: {bwd_bw:.2f} GB/s")
```

```
Eager forward median time:  3.0577 ms
Eager backward median time: 3.7642 ms
Eager forward achieved BW:  343.63 GB/s
Eager backward achieved BW: 558.26 GB/s
```

(c) Benchmark torch.compile mode. (10 points)

(i) Wrap the cross-entropy computation with torch.compile and repeat the benchmarking from part (b). Note that the first call triggers compilation — make sure your warmup accounts for this. Report the achieved memory bandwidth. (4 points)

(ii) How does torch.compile compare to eager mode and to your ideal prediction? Explain what torch.compile is likely doing differently (e.g., kernel fusion) that accounts for the performance difference. (6 points)

(i) Using the compiled version:

- Compiled forward time: $1.6824\ ms$
- Compiled backward time: $1.6742\ ms$

the achieved bandwidths are:

- Forward BW = $(\frac{1.0507}{1.6824/1000}) \approx 624.52\ GB/s$
- Backward BW = $(\frac{2.1014}{1.6742/1000}) \approx 1255.16\ GB/s$

(ii) Comparison with eager mode(A) and the ideal prediction(B):

A. Compared to eager mode:

Eager forward: 3.0577 ms ---> Compiled forward: 1.6824 ms

Eager backward: 3.7642 ms ---> Compiled backward: 1.6742 ms

- Forward speedup: $\frac{3.0577}{1.6824} \approx 1.82\times$

- Backward speedup: $\frac{3.7642}{1.6742} \approx 2.25\times$

Hence, *torch.compile* is much faster than eager mode.

B. Compared to the ideal prediction from part (a):

Ideal forward: 0.515 ms ---> Compiled forward: 1.6824 ms

Ideal backward: 1.031 ms ---> Compiled backward: 1.6742 ms

- So *torch.compile* gets closer to the ideal than eager mode, but it is still slower than the ideal values.

Explain what torch.compile is likely doing differently (e.g., kernel fusion) that accounts for the performance difference.

- A likely reason is that *torch.compile* can fuse operations into fewer kernels and reduce unnecessary memory traffic. Instead of materializing as many intermediate tensors in HBM, it can keep more intermediate values in on-chip memory and reduce kernel launch overhead. Since cross-entropy is memory-bound, this reduction in memory movement is the main reason for the performance improvement.

Code for part (c):

```
def ce_forward_fn(logits, targets):
    return F.cross_entropy(logits, targets, reduction='none')

compiled_ce_forward_fn = torch.compile(ce_forward_fn)

def benchmark_ce_compiled(logits, targets, iters=100, warmup=10):
    fwd_times = []
    bwd_times = []

    # Warmup (also triggers compilation)
    for _ in range(warmup):
        loss = compiled_ce_forward_fn(logits, targets)
        grad_out = torch.ones_like(loss, dtype=torch.float32)
        loss.backward(grad_out, retain_graph=False)
        logits.grad = None
    torch.cuda.synchronize()

    # Forward benchmark
    for _ in range(iters):
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)

        start.record()
        loss = compiled_ce_forward_fn(logits, targets)
        end.record()

        torch.cuda.synchronize()
        fwd_times.append(start.elapsed_time(end))  # ms

    # Backward benchmark
    for _ in range(iters):
        loss = compiled_ce_forward_fn(logits, targets)
        grad_out = torch.ones_like(loss, dtype=torch.float32)
```

```
        logits.grad = None
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)

        start.record()
        loss.backward(grad_out, retain_graph=False)
        end.record()

        torch.cuda.synchronize()
        bwd_times.append(start.elapsed_time(end))  # ms

    fwd_median_ms = statistics.median(fwd_times)
    bwd_median_ms = statistics.median(bwd_times)

    return fwd_median_ms, bwd_median_ms, fwd_times, bwd_times


# Run benchmark
compiled_fwd_ms, compiled_bwd_ms, compiled_fwd_times, compiled_bwd_times = benchmark_ce_compiled(
    logits, targets, iters=100, warmup=10
)

compiled_fwd_bw = BYTES_FWD_GB / (compiled_fwd_ms / 1000.0)
compiled_bwd_bw = BYTES_BWD_GB / (compiled_bwd_ms / 1000.0)

print(f"Compiled forward median time:  {compiled_fwd_ms:.4f} ms")
print(f"Compiled backward median time: {compiled_bwd_ms:.4f} ms")
print(f"Compiled forward achieved BW:  {compiled_fwd_bw:.2f} GB/s")
print(f"Compiled backward achieved BW: {compiled_bwd_bw:.2f} GB/s")
```

```
Compiled forward median time:  1.6824 ms
Compiled backward median time: 1.6742 ms
Compiled forward achieved BW:  624.51 GB/s
Compiled backward achieved BW: 1255.14 GB/s
```

(d) Inspecting the compiled kernel. (10 points)

When torch.compile compiles your function, it generates Triton kernel code under the hood.

(i) Use the environment variable TORCH LOGS=output code or torch. dynamo.config.log level to dump the generated Triton kernel code for the cross-entropy forward pass. Include the generated code (or relevant excerpts) in your notebook. (3 points)

(ii) Annotate the generated kernel: identify which lines correspond to (1) computing the max for numer- ical stability, (2) the exp and sum (softmax denominator), (3) the log-sum-exp, and (4) the final loss computation. Relate these back to the formulas you derived in Theory Problem 1(a). (7 points)

(i) I enabled compiler code dumping and inspected the generated TorchInductor files under /tmp/torchinductor_root. The file I found (idx: 3) for the forward pass is labeled # AOT ID: ['0_forward'], and it shows the forward cross-entropy computation built from *aten._log_softmax, prims.prepare_softmax_online, aten.sub,* and *aten.nll_loss_forward*

```
# A expert of the generated Triton Kernal as follows, full version is included in the below "code for part(d)" section.
_tmp3_max = tl.full([XBLOCK, R0_BLOCK], float('-inf'), tl.float32)
_tmp3_sum = tl.zeros([XBLOCK, R0_BLOCK], tl.float32)
for r0_offset in range(0, r0_numel, R0_BLOCK):
```

```
        ...
        tmp0 = tl.load(in_ptr0 + (r0_1 + 128256*x0), r0_mask, eviction_policy='evict_last', other=0.0).to(tl.float32)
        ...
        _tmp3_max_next, _tmp3_sum_next = triton_helpers.online_softmax_combine(
            _tmp3_max, _tmp3_sum, tmp2, False
        )
        _tmp3_max = tl.where(r0_mask, _tmp3_max_next, _tmp3_max)
        _tmp3_sum = tl.where(r0_mask, _tmp3_sum_next, _tmp3_sum)

    tmp3, tmp4 = triton_helpers.online_softmax_reduce(
        _tmp3_max, _tmp3_sum, 1, False)
    ...
    tmp5 = tl_math.log(tmp4)
    ...
    tmp16 = tl.load(in_ptr0 + (tmp14 + 128256*x0), None, eviction_policy='evict_last').to(tl.float32)
    tmp18 = tmp17 - tmp3
    tmp19 = tmp18 - tmp5
    tmp21 = -tmp20
    tmp23 = tl.where(tmp8, tmp21, tmp22)
```

This kernel seems to compute the stable softmax statistics row-by-row and then forms the per-token cross-entropy loss

(ii) Annotating the generated kernel

From Theory 1(a), the stable cross-entropy formula is:

$$\ell(x, y) = -x_y + m + \log\left(\sum_{j=1}^{V} e^{x_j - m}\right), \qquad m = \max_j x_j$$

1. Computing the max for numerical stability

These lines initialize and update the running row-wise maximum:

```
    _tmp3_max = tl.full([XBLOCK, R0_BLOCK], float('-inf'), tl.float32)
    ...
    _tmp3_max_next, _tmp3_sum_next = triton_helpers.online_softmax_combine(
        _tmp3_max, _tmp3_sum, tmp2, False
    )
    ...
    tmp3, tmp4 = triton_helpers.online_softmax_reduce(
        _tmp3_max, _tmp3_sum, 1, False)
```

Here, `tmp3` is the final row-wise maximum $m = max_j\ x_j$.

The kernel uses an online softmax reduction, so it computes the max while going through the vocabulary dimension instead of materializing a full intermediate tensor.

2. The exp and sum (softmax denominator)

These same online softmax lines also accumulate the denominator information-

```
    _tmp3_sum = tl.zeros([XBLOCK, R0_BLOCK], tl.float32)
    ...
```

```
_tmp3_max_next, _tmp3_sum_next = triton_helpers.online_softmax_combine(
    _tmp3_max, _tmp3_sum, tmp2, False
)
...
tmp3, tmp4 = triton_helpers.online_softmax_reduce(
    _tmp3_max, _tmp3_sum, 1, False)
```

Here, `tmp4` is the accumulated softmax denominator term

$$\sum_{j=1}^{V} e^{x_j - m}$$

So, the max and the denominator are computed together using the online softmax helper.

3. The log-sum-exp

This line computes the logarithm of the denominator:

```
tmp5 = tl_math.log(tmp4)
```

So `tmp5` is,

$$log(\sum_{j=1}^{V} e^{x_j - m})$$

Together with `tmp3` = m, this gives the stable log-sum-exp term,

$$m + log(\sum_{j=1}^{V} e^{x_j - m})$$

4. Final loss computation

These lines load the target logit and compute the final loss:

```
tmp16 = tl.load(in_ptr0 + (tmp14 + 128256*x0), None, eviction_policy='evict_last').to(tl.float32)
tmp17 = tmp16.to(tl.float32)
tmp18 = tmp17 - tmp3
tmp19 = tmp18 - tmp5
tmp20 = tmp19.to(tl.float32)
tmp21 = -tmp20
tmp23 = tl.where(tmp8, tmp21, tmp22)
```

$$x_y - m - \log\left(\sum_{j=1}^{V} e^{x_j - m}\right)$$

- x_y = `tmp17`
- m = `tmp3`
- $\log(\sum e^{x_j - m})$ = `tmp5`

So:

- `tmp18 = tmp17 - tmp3`

- `tmp19 = tmp18 - tmp5` : gives the required formula.

Negation

$$\ell(x, y) = -\left(x_y - m - \log\left(\sum_{j=1}^{V} e^{x_j - m}\right)\right)$$

- `tmp20 = tmp19`
- `tmp21 = -tmp20` : negated

Final form

$$\ell(x, y) = -x_y + m + \log\left(\sum_{j=1}^{V} e^{x_j - m}\right)$$

- This is `tmp21`

Code for part (d):

```
# enable logging and compile the forward function
import os
import torch._dynamo
import torch._logging

os.environ["TORCH_COMPILE_DEBUG"] = "1"

torch._dynamo.reset()
torch._logging.set_logs(output_code=True, kernel_code=True)

def ce_forward_only(logits, targets):
    return F.cross_entropy(logits, targets, reduction="none")

compiled_ce_forward_only = torch.compile(
    ce_forward_only,
    backend="inductor",
    fullgraph=True
)

# trigger compilation
out = compiled_ce_forward_only(logits, targets)
torch.cuda.synchronize()

print("Done. Output shape:", out.shape)
```

```
W0410 22:55:17.394000 2120 torch/_logging/_internal.py:475] Using TORCH_LOGS environment variable for log settings, ignoring call to set_logs
Done. Output shape: torch.Size([4096])
```

```
# list generated code files
candidate_files = []

for root, dirs, files in os.walk("/tmp"):
    for f in files:
        full = os.path.join(root, f)
```

```
        if (
            ("torch_compile_debug" in full or "inductor" in full or "triton" in full)
            and f.endswith((".py", ".tt", ".cu", ".txt"))
        ):
            candidate_files.append(full)

print("Found", len(candidate_files), "candidate files:\n")
for i, path in enumerate(candidate_files[:200]):
    print(f"[{i}] {path}")
```

```
Found 4 candidate files:

[0] /tmp/torchinductor_root/lv/clvvyellm6jzia7xkttdw5kidot4zsxxpqbat6dnijjvgispr52d.py
[1] /tmp/torchinductor_root/s3/cs347yqcryxc5wibae327gidjnn7xlwmta4bsvzwapcqkykalola.py
[2] /tmp/torchinductor_root/qk/cqkaoohneey6hdkreuuxt5zv3eh6z66jws3k7nrsnek4rmugfotb.py
[3] /tmp/torchinductor_root/zf/czfuppc55twg4qsglisr7jhfthxnjer3v6fhxcnjmln2vozyzslu.py
```

```
# open one generated file
idx = 3 # found 3 as the forward kernal

path = candidate_files[idx]
print("Opening:", path)

with open(path, "r") as f:
    text = f.read()

print(text[:20000])
```

```
            buf3 = empty_strided_cuda((4096, ), (1, ), torch.bfloat16)
            # Topologically Sorted Source Nodes: [cross_entropy], Original ATen: [aten._log_softmax, prims.prepare_softmax_online, aten.sub, aten.nll_loss_forward]
            stream0 = get_raw_stream(0)
            triton_red_fused__log_softmax_nll_loss_forward_prepare_softmax_online_sub_0.run(buf2, primals_1, primals_2, buf0, buf3, 4096, 128256, stream=stream0)
        return (buf3, primals_1, primals_2, buf0, buf2, )

runner = Runner(partitions=[])
call = runner.call
recursively_apply_fns = runner.recursively_apply_fns


def benchmark_compiled_module(times=10, repeat=10):
    from torch._dynamo.testing import rand_strided
    from torch._inductor.utils import print_performance
    primals_1 = rand_strided((4096, 128256), (128256, 1), device='cuda:0', dtype=torch.bfloat16)
    primals_2 = rand_strided((4096, ), (1, ), device='cuda:0', dtype=torch.int64)
    fn = lambda: call([primals_1, primals_2])
    return print_performance(fn, times=times, repeat=repeat)


if __name__ == "__main__":
    from torch._inductor.wrapper_benchmark import compiled_module_main
    compiled_module_main('None', benchmark_compiled_module)
```

```
# search inside the opened file for useful lines
keywords = ["amax", "max", "exp", "sum", "log", "nll_loss", "softmax", "log_softmax"]

lines = text.splitlines()
for i, line in enumerate(lines):
    if any(k in line for k in keywords):
        print(f"{i+1:4d}: {line}")
```

```
  38: # Topologically Sorted Source Nodes: [cross_entropy], Original ATen: [aten._log_softmax, prims.prepare_softmax_online, aten.sub, aten.nll_loss_forward]
  40: #   cross_entropy => convert_element_type, convert_element_type_1, full_default, full_default_1, gather, log, ne, neg, squeeze, sub_1, unsqueeze, where, where_1
  46: #   %log : Tensor "f32[4096, 1][1, 1]cuda:0" = PlaceHolder[target=log]
  48: #   %prepare_softmax_online_default : [num_users=2] = call_function[target=torch.ops.prims.prepare_softmax_online.default](args = (%convert_element_type, 1), kwargs = {})
  50: #   %log : Tensor "f32[4096, 1][1, 1]cuda:0"[num_users=2] = call_function[target=torch.ops.aten.log.default](args = (%getitem_1,), kwargs = {})
  51: #   %sub_1 : Tensor "f32[4096, 128256][128256, 1]cuda:0"[num_users=1] = call_function[target=torch.ops.aten.sub.Tensor](args = (%sub_tensor, %log), kwargs = {})
  62: #   return %getitem,%getitem_1,%log,%where_1
  63: triton_red_fused__log_softmax_nll_loss_forward_prepare_softmax_online_sub_0 = async_compile.triton('triton_red_fused__log_softmax_nll_loss_forward_prepare_softmax_online_sub_0
  76:     triton_meta={'signature': {'in_out_ptr0': '*fp32', 'in_ptr0': '*bf16', 'in_ptr1': '*i64', 'out_ptr0': '*fp32', 'out_ptr1': '*bf16', 'xnumel': 'i32', 'r0_numel': 'i32', 'XB
  77:     inductor_meta={'grid_type': 'Grid1D', 'autotune_hints': set(), 'kernel_name': 'triton_red_fused__log_softmax_nll_loss_forward_prepare_softmax_online_sub_0', 'mutated_arg_n
  80: def triton_red_fused__log_softmax_nll_loss_forward_prepare_softmax_online_sub_0(in_out_ptr0, in_ptr0, in_ptr1, out_ptr0, out_ptr1, xnumel, r0_numel, XBLOCK : tl.constexpr, R0_
  84:     RBLOCK: tl.constexpr = R0_BLOCK
  91:     _tmp3_max = tl.full([XBLOCK, R0_BLOCK], float('-inf'), tl.float32)
  92:     _tmp3_sum = tl.zeros([XBLOCK, R0_BLOCK], tl.float32)
 103:         _tmp3_max_next, _tmp3_sum_next = triton_helpers.online_softmax_combine(
 104:             _tmp3_max, _tmp3_sum, tmp2, False
 107:         _tmp3_max = tl.where(r0_mask, _tmp3_max_next, _tmp3_max)
 108:         _tmp3_sum = tl.where(r0_mask, _tmp3_sum_next, _tmp3_sum)
 110:     tmp3, tmp4 = triton_helpers.online_softmax_reduce(
 111:         _tmp3_max, _tmp3_sum, 1, False)
 116:     tmp5 = tl_math.log(tmp4)
 164:             # Topologically Sorted Source Nodes: [cross_entropy], Original ATen: [aten._log_softmax, prims.prepare_softmax_online, aten.sub, aten.nll_loss_forward]
 166:             triton_red_fused__log_softmax_nll_loss_forward_prepare_softmax_online_sub_0.run(buf2, primals_1, primals_2, buf0, buf3, 4096, 128256, stream=stream0)
```

Problem 2: Bonus — Cross-Entropy Speed Competition

See task specification and submission instruction in the problem set PDF. Submission window is open from April 7 to April 16. If you're submitting for bonus credits, please include your optimization journal here documenting your optimization process:

- What approaches did you try? (e.g., torch.compile tricks, custom Triton kernels, fused forward+backward, memory layout changes, block size tuning, etc.)
- For each approach, report the timing and explain why it was faster or slower than your previous best.
- What is the achieved memory bandwidth of your final submission, and what fraction of peak A100 bandwidth (2039 GB/s) does it reach? What do you think is preventing you from reaching 100%?

If you prefer to document your optimization in a separate PDF, please also make a note here.

Initial checks are as follows. This was attempted just to check whether the files ran or not. Further modified "submission.py" file and journal are atatched with the submission to Gradescope.

GitHub - TharakaDFonseka

```
print(torch.__version__)
print(torch.cuda.get_device_name(0))
```

```
2.10.0+cu128
NVIDIA A100-SXM4-40GB
```

```
import importlib.util
import sys

path = "/content/submission.py"

spec = importlib.util.spec_from_file_location("submission", path)
submission = importlib.util.module_from_spec(spec)
sys.modules["submission"] = submission
spec.loader.exec_module(submission)
```

```
B, V = 4096, 128256
logits = torch.randn(B, V, device="cuda", dtype=torch.bfloat16)
targets = torch.randint(0, V, (B,), device="cuda", dtype=torch.int64)
grad_output = torch.randn(B, device="cuda", dtype=torch.float32)
```

```
losses = submission.cross_entropy_forward(logits, targets)
grad_logits = submission.cross_entropy_backward(logits, targets, grad_output)

print(losses.shape, losses.dtype)
print(grad_logits.shape, grad_logits.dtype)
```

```
torch.Size([4096]) torch.bfloat16
torch.Size([4096, 128256]) torch.bfloat16
```

```
import torch.nn.functional as F

# reference forward
ref_losses = F.cross_entropy(logits, targets, reduction="none")
```

```
# reference backward
x = logits.float()
probs = torch.softmax(x, dim=1)
rows = torch.arange(targets.shape[0], device=targets.device)
probs[rows, targets] -= 1.0
ref_grad = probs * grad_output[:, None]
ref_grad = ref_grad.to(torch.bfloat16)

print("forward allclose:", torch.allclose(losses, ref_losses, atol=1e-3, rtol=1e-2))
print("backward allclose:", torch.allclose(grad_logits, ref_grad, atol=1e-3, rtol=1e-2))
```

```
forward allclose: True
backward allclose: True
```

```
# reference forward
ref_losses = F.cross_entropy(logits, targets, reduction="none")

# reference backward
x = logits.float()
probs = torch.softmax(x, dim=1)
rows = torch.arange(targets.shape[0], device=targets.device)
probs[rows, targets] -= 1.0
ref_grad = probs * grad_output[:, None]
ref_grad = ref_grad.to(torch.bfloat16)

print("forward allclose:", torch.allclose(losses, ref_losses, atol=1e-3, rtol=1e-2))
print("backward allclose:", torch.allclose(grad_logits, ref_grad, atol=1e-3, rtol=1e-2))
```

```
forward allclose: True
backward allclose: True
```

```
import statistics

def bench_fn(fn, iters=50, warmup=10):
    times = []
    for _ in range(warmup):
        fn()
    torch.cuda.synchronize()

    for _ in range(iters):
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)
        start.record()
        fn()
        end.record()
        torch.cuda.synchronize()
        times.append(start.elapsed_time(end))
    return statistics.median(times)

fwd_ms = bench_fn(lambda: submission.cross_entropy_forward(logits, targets))
bwd_ms = bench_fn(lambda: submission.cross_entropy_backward(logits, targets, grad_output))

print("forward ms:", fwd_ms)
print("backward ms:", bwd_ms)
```

```
forward ms: 1.7285120487213135
backward ms: 6.390783786773682
```

```
%cd /content
!python test_cross_entropy.py submission_1.py
!python test_cross_entropy.py submission_2.py
!python test_cross_entropy.py submission.py
```

```
V0411 00:38:29.585000 59465 torch/_inductor/graph.py:2469] [0/0] [__output_code]
```

```
from google.colab import files
uploaded = files.upload()   # In the file picker, select all required .py files from step 1
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving colab_benchmark_cross_entropy.py to colab_benchmark_cross_entropy.py
Saving submission_2.py to submission_2.py
Saving submission_3.py to submission_3.py
Saving test_cross_entropy.py to test_cross_entropy.py
```

```
import os
os.environ.pop("TORCH_LOGS", None)
```

```
''
```

```
%cd /content
!python colab_benchmark_cross_entropy.py
```

```
      V |   Fwd ms   Bwd ms  Fwd+Bwd ms |     Fwd BW     Bwd BW  Fwd+Bwd BW |  Fwd GB  Bwd GB
-----------------------------------------------------------------------------------------------

Benchmarking baseline (PyTorch eager)...

      V |   Fwd ms   Bwd ms  Fwd+Bwd ms |     Fwd BW     Bwd BW  Fwd+Bwd BW |  Speedup
-----------------------------------------------------------------------------------------
  32000 |   0.435    0.843       1.220  |    602.5     622.2      644.9  |    4.67x
  50264 |   0.758    1.645       2.339  |    543.5     500.8      528.2  |    3.90x
 128256 |   1.541    3.165       4.642  |    682.0     663.9      679.0  |    5.01x

Units: time in ms, bandwidth in GB/s

** Competition score (geomean speedup vs baseline): 4.50x **
```

LLM Prompts

If you used an AI tool to complete any part of this assignment, please paste all prompts you used to produce your final code/responses in the box below and answer the following reflection question.

Prompts Used:

- A. Compared to ideal forward and backward of o.51 and 1.3 ms to calculate cross entropy o.51 and 1.3 ms, eager mode gives forward 3ms and backward 3.7ms is this okay or too slow?
- B. What is torch.compile mean?
- C. how to dump triton kernel from torch.compile? TORCH_LOGS not printing anything
- D. what is a triton kernel?
- E. I have 4 files, how to find which is forward?

Reflection: What parts of the AI generated output required modification or improvement? Describe the feedback you gave the tool to produce your final output or any changes you had to make on your own.

- C.Had to go back and forth few times (where are generated kernels stored? etc.) to get the understanding of enabling compiler code dumping and inspected the generated TorchInductor files under /tmp/torchinductor_root.

Start coding or generate with AI.